



NAME : HAMDAN ALI

WEEK 4 : Day 18

DAY 18

Build a form that validates empty inputs before submission.

Task 18 : Screenshoots Of Output

DS

DEVSUMMIT '26

Three days.
One stage.
Every engineer who
ships for a living.

Dates

August 12–14, 2026

Venue

Lower Chitral Near Ataliq Bridge , Pakistan

Track

Full-Stack & Platform Engineering

YOUR BOARDING CHECKLIST

☐ Full name

☐ Work email

☐ Phone number

☐ Organization

☐ Role

☐ Password set

☐ Terms accepted

Passes are non-transferable and validated at the door against the email listed here.

ATTENDEE REGISTRATION

Reserve your seat

Every field below is required — we validate as you go, so there are no surprises at submit.

Full name

Hamdan Ali

Work email

Hamdanaly651@gmail.com

Phone number

03XXX XXXXXXXX

Organization

Company or university

Which best describes you?

Select a role

Password

At least 8 characters

Confirm password

Re-enter password

☐ I agree to the code of conduct and the attendee terms.

Confirm registration →

BOARDING CONFIRMED

DS26-888888

You're on the list.

ATTENDEE

—

ORGANIZATION

—

VENUE

Lower Chitral Near Ataliq Bridge , Pakistan

EMAIL

—

ROLE

—

DATES

August 12–14, 2026

Edit details



Which best describes you?

Select a role

Select a role

Frontend Engineer

Backend Engineer

Full-Stack Engineer

Student / Intern

Other

After Filling The Information:

ATTENDEE REGISTRATION

Reserve your seat

Every field below is required – we validate as you go, so there are no surprises at submit.

Full name

Hamdan Ali

Work email

hamdanaly651@gmail.com

Phone number

03497617197

Organization

Air University , Islamabad

Which best describes you?

Full-Stack Engineer

Password

Confirm password

☒ I agree to the code of conduct and the attendee terms.

Confirm registration →

Result:

DS DEVSUMMIT '26

Three days.
One stage.
Every engineer who
ships for a living.

Dates August 12–14, 2026

Venue Lower Chitral Near Ataliq Bridge , Pakistan

Track Full-Stack & Platform Engineering

YOUR BOARDING CHECKLIST

✓ Full name

✓ Work email

✓ Phone number

✓ Organization

✓ Role

✓ Password set

✓ Terms accepted

Passes are non-transferable and validated at the door against the email used here.

BOARDING CONFIRMED

DS26-655183

You're on the list.

ATTENDEE
Hamdan Ali

ORGANIZATION
Air University , Islamabad

VENUE
Lower Chitral Near Ataliq Bridge , Pakistan

EMAIL
hamdanaly651@gmail.com

ROLE
Full-Stack Engineer

DATES
August 12–14, 2026

Barcode

Edit details



EXPLANATION

DAY 18 — FORM HANDLING & VALIDATION

Building the Boarding Checklist

Hamdan Ali · Week 4 · Full Stack Web Development Internship

What I Was Going For

The task itself was simple on paper: build a form that checks for empty inputs before it lets someone submit. I could have made one text field and one "Please fill this in" message and called it done. Instead I wanted to build something that actually felt like a product a real event would ship — a conference registration form, with enough fields that empty-input validation actually matters, and enough visual identity that it doesn't look like a form exercise.

I settled on a fictional developer conference called DevSummit '26, and built the registration flow around a boarding-pass idea — you're checking in for a seat, the same way you'd check in for a flight. That framing gave me a natural excuse to make validation visible and satisfying instead of just functional: as each field becomes valid, it ticks off on a live checklist sitting next to the form, so filling out the form and watching your checklist fill up feel like the same action.

Structuring the Page

The page is split into two panels. The left one is dark, holds the event branding, the event details, and the live checklist — it doesn't change once the page loads except for that checklist. The right one is light, and holds the actual form. Between them there's a thin dashed seam with little circular notches cut out of both edges, styled to look like the perforation on a real ticket stub, which was a small detail but it's the one piece of the design I spent the most time getting right, since it's what actually sells the "boarding pass" idea before anyone reads a single word.

On submit, if every field passes validation, the form itself is hidden and replaced with a confirmation ticket built from whatever the person just typed in — their name, email, organization,

and role, plus a confirmation code generated on the spot. It's a small piece of theatre, but it also proves the validation actually worked, rather than just saying "success" and moving on.

How the Validation Is Actually Structured

Rather than writing one long if/else chain checking every field inside the submit handler, I put every field's rule in its own small function, all sitting together in one validators object. Each one takes the field's current value and returns an error string if something's wrong, or an empty string if it's fine:

```
const validators = {
  fullName: (value) => {
    const trimmed = value.trim();
    if (!trimmed) return 'Please enter your full name.';
    if (trimmed.length < 2) return 'That name looks a little short.';
    return '';
  },

  email: (value) => {
    const trimmed = value.trim();
    if (!trimmed) return 'An email address is required.';
    const emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
    if (!emailPattern.test(trimmed)) return 'Enter a valid email, like name@company.com.';
    return '';
  },

  // ...phone, organization, role, password, confirmPassword, terms
};
```

The task only asked for empty-field checking, and every rule above does start with exactly that — an empty check first, before anything else. But once I had the article on JavaScript form validation open, it felt wrong to stop at "is it empty" for a field like email or password, when a genuinely empty email and a genuinely malformed one are both going to cause the same problem for whoever processes this form later. So each field checks for emptiness first, and a few of them go one step further into checking the shape of what was actually typed.

Confirm-password is the one rule that depends on another field rather than just itself — it checks against whatever is currently sitting in the password field, which meant I also had to re-run its validation any time the password field changed, not just when confirm-password itself changed. That was the one genuinely tricky piece of the whole form.

Making Validation Happen Live, Not Just on Submit

This is the part that came straight from the two articles. The tutorial content covers handling form events in general, and the W3Schools page specifically walks through validating on submit versus validating as the user types. I ended up using both: every field validates when the user leaves it (blur), and once a field has shown an error, it re-validates on every keystroke after that, so the red border and the message disappear the moment the input becomes valid instead of sitting there until the next submit attempt:

```
el.addEventListener('blur', () => validateField(name));

el.addEventListener(eventName, () => {
  const fieldEl = form.querySelector(`[data-field="${name}"]`);
  if (fieldEl.classList.contains('is-invalid') || el.type === 'checkbox' || el.tagName === 'SELECT') {
    validateField(name);
  }
});
```

I didn't want every field flashing red the instant someone starts typing their name — that's a genuinely annoying pattern I've hit on other sites. Validating on blur first, and only going live-as-you-type after an error has already appeared once, felt like the right middle ground between "tells you nothing until you hit submit" and "yells at you before you've finished typing."

The Checklist Is Just the Same State, Rendered Twice

The boarding checklist on the left isn't a separate system tracking its own state — it reads off the exact same validation result the form fields use. Every time a field is validated, one function updates that field's border and error text, and calls a second function that finds the matching checklist row and toggles a single class on it:

```
function updateChecklist(name, isValid) {  
  const item = document.querySelector(`.checklist__item[data-check="${name}"]`);  
  if (!item) return;  
  const hasValue = name === 'terms'  
    ? fields.terms.checked  
    : readValue(name).toString().trim() !== "";  
  item.classList.toggle('is-done', isValid && hasValue);  
}
```

That's the whole mechanism — one boolean, one class toggle, and CSS handles the rest, turning the hollow dot into a filled checkmark with a small pop animation. It would have been easy to build this as two separate things that both needed to be kept in sync by hand, and I'm glad I caught that early, because keeping it to one source of truth means there's no way for the checklist and the form to disagree with each other.

Refusing to Submit, Visibly

On submit, the handler calls `preventDefault()` immediately, then runs every field through `validateField()` regardless of whether the person touched it or not — so tabbing straight to the submit button without filling anything in still catches every empty field, not just the ones that happened to lose focus already:

```
form.addEventListener('submit', (event) => {  
  event.preventDefault();  
  const results = Object.keys(fields).map((name) => validateField(name));  
  const allValid = results.every(Boolean);  
  
  if (!allValid) {  
    form.classList.add('is-shaking');  
    const firstInvalid = form.querySelector('.is-invalid input, .is-invalid select');  
    if (firstInvalid) firstInvalid.focus();  
    return;  
  }  
  showTicket();  
});
```

If anything fails, the form gives a short shake and moves focus to the first invalid field, rather than just leaving a wall of red text for the person to hunt through on their own. The video on form handling was useful here specifically for that idea — that rejecting a submission should point somewhere, not just refuse silently.

A Few Notes on What I Took From This

- Checking for empty input is the floor, not the whole job — the moment a field has an expected shape, like an email or a password length, it's worth checking that too, using the exact same pattern as the empty check.
- Validating on blur first, then live once an error has shown, felt noticeably better to actually use than validating on every keystroke from the start — small UX decisions like this are easy to skip but change how the form feels a lot.
- Driving two visual elements — the field's own error state and the checklist — off one shared validation result, instead of tracking them separately, removed a whole category of bugs before they could happen.
- `preventDefault()` combined with re-validating every field on submit, not just the ones already touched, is what actually makes a form "safe" — relying only on blur events would let someone tab past a field and never trigger its check at all.

References Used

I used the TutorialsPoint page on JavaScript form handling for the general structure of listening to form events, and the W3Schools validation article for the specific difference between checking on submit and checking as the user types. Both videos were useful for seeing form validation applied to a real, working example rather than an isolated snippet.

Article: [tutorialspoint.com/javascript/javascript_form_handling.htm](https://www.tutorialspoint.com/javascript/javascript_form_handling.htm)

Article: [w3schools.com/js/js_validation.asp](https://www.w3schools.com/js/js_validation.asp)

Video: youtube.com/watch?v=tLBlhp0SA_0

Video: youtube.com/watch?v=WLUCzSaH5kI



Files Submitted

- index.html — the two-panel layout, the registration form, and the confirmation ticket markup.
- style.css — the full visual system: the branding panel, the ticket-stub perforation, field states, and the checklist animation.
- script.js — the validators object, live validation wiring, the checklist sync, and the submit handling described above.

THANK YOU!